3: SCENARIO: COLLEGE CLUB MEMBERSHIP MANAGEMENT

A college runs several student clubs such as Music Club, Sports Club, and Coding Club. The college currently stores all club-related information in one table, shown below.

Student ID	Student-Name	Email	ClubName	ClubRoom	ClubMentor	JoinDate
1	Asha	asha@email.com	Music Club	R101	Mr. Raman	1/10/2024
2	Bikash	bikash@email.com	Sports Club	R202	Ms. Sita	1/12/2024
1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	1/15/2024
3	Nisha	nisha@email.com	Music Club	R101	Mr. Raman	1/20/2024
4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	1/18/2024
5	Suman	suman@email.com	Music Club	R101	Mr. Raman	1/22/2024
2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	1/25/2024
6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	1/27/2024
3	Nisha	nisha@email.com	Coding Club	Lab1	Mr. Anil	1/28/2024
7	Aman	aman@email.com	Coding Club	Lab1	Mr. Anil	1/30/2024

3.1: IDENTIFY DATA PROBLEMS

3.1.1 PROBLEMS IN THE GIVEN TABLE.

Problem 1: Redundant (repeated) student data
The same student's name and email are repeated every time they join a different club.
Example: Asha (StudentID 1) appears in Music Club and Sports Club, so her details repeat.

Problem 2: Redundant (repeated) club data
Club details like ClubRoom and ClubMentor repeat for every student in that club.
Example: Music Club always repeats R101, Mr. Raman in multiple rows.

Problem 3: Update Anomaly
If a club mentor changes (e.g., Sports Club mentor changes), you must update multiple rows. If you miss one row, the table becomes inconsistent.

3.1.2 REASON THE TABLE IS NOT NORMALIZED

This table is not normalized because it keeps different kinds of information in one single table. It includes student details like StudentName and Email, club details like ClubRoom and ClubMentor, and also membership information like JoinDate. When all this information is stored together, the same data gets repeated many times. For example, if a student joins more than one club, their name and email appear again in another row. In the same way, the club room and mentor are repeated for every student who joins that club. This repetition creates unnecessary data and can cause problems when inserting, updating, or deleting records. In a properly normalized database, student details would be stored in one table, club details in another table, and the membership information in a separate table that connects students with clubs.

3. Define:

a. Redundant data and duplicate data

Redundant data:

Data that is repeated unnecessarily due to poor table design.
Example: “Music Club, R101, Mr. Raman” repeated for every student.

Duplicate data:

When the same record appears more than once (often by mistake).
Example: If the same student membership row is entered twice accidentally.

b. Insert anomaly, update anomaly, and deletion anomaly.

Insert anomaly:
You cannot add some information unless you also add other unrelated information.
Example: You can’t add a club unless a student joins it.

Update anomaly:
The same detail must be updated in many places, and missing one update causes inconsistent data.
Example: Changing a club room requires updating multiple rows.

Deletion anomaly:
Deleting a row removes important data you still need.
Example: Deleting the last member of a club deletes the club’s details too.

3.2: NORMALIZE THE DATA

3.2.1 ATOMIC VALUES IN FIELDS

Normalization is a method used in databases to organize data properly. The main goal of normalization is to reduce repeated data and make the database easier to manage. Instead of keeping everything in one large table, the data is divided into smaller tables that are connected to each other. This helps avoid errors and keeps the data more accurate. Normalization is usually done in steps called normal forms. (Elmasri & Navathe, 2016)

Step 1: First Normal Form (1NF)

First Normal Form means that every column in a table should contain only one value, and the table should not have repeating groups. Each row should also be able to be identified with a primary key.

In the given table, each column already has a single value. For example, every row contains one student name, one email, and one club name. This means the table follows the rule of atomic values and does not have repeating groups in a single column.

However, StudentID alone cannot be used as the primary key because one student can join more than one club. Because of this, a composite primary key is needed. The combination of StudentID and ClubName can be used together to uniquely identify each record in the table.

StudentID	Student Name	Email	ClubName	ClubRoom	ClubMentor	JoinData
1	Asha	asha@email.com	Music Club	R101	Mr.Raman	1/10/2024
1	Asha	asha@email.com	Sports Club	R202	Ms.Sita	1/15/2024
2	Bikash	bikash@email.com	Sports Club	R202	Ms.Sita	1/12/2024
2	Bikash	bikash@email.com	Drama Club	R303	Mr.Kiran	1/25/2024
3	Nisha	nisha@email.com	Music Club	R101	Mr.Raman	1/20/2025
3	Nisha	nisha@email.com	Coding Club	Lab1	Mr.Anil	1/28/2025
4	Rohan	rohan@email.com	Drama Club	R303	Mr.Kiran	1/18/2024
5	Simran	simran@email.com	Music Club	R101	Mr.Raman	1/22/2024
6	Pooja	pooja@email.com	Sports Club	R202	Ms.Sita	1/27/2024
7	Aman	aman@email.com	Coding Club	Lab1	Mr.Anil	1/30/2024

Table 1: Original club membership table showing repeated student and club data.

Step 2: Second Normal Form (2NF)

Second Normal Form means the table must already be in 1NF and all the columns should depend on the whole primary key. There should not be partial dependency.

In this table, the primary key is a combination of StudentID and ClubName. However, some columns depend only on part of this key. For example, StudentName and Email depend only on StudentID, while ClubRoom and ClubMentor depend only on ClubName. This is called partial dependency and it causes repeated data.

To fix this problem, the table is divided into three separate tables: Student, Club, and Membership. This helps organize the data better and reduces repetition.

Tables in Second Normal Form (2NF)

Student Table:

Student (PK)	Student Name	Email
1	Asha	asha@email.com
2	Bikash	bikash@email.com
3	Nisha	nisha@email.com
4	Rohan	rohan@email.com
5	Suman	suman@email.com
6	Pooja	pooja@email.com
7	Aman	aman@email.com

Table 2 Student table after normalization (2NF)

Club Table:

ClubName (PK)	Club Room	Club Mentor
Music Club	R101	Mr.Raman
Sports Club	R202	Ms.Sita
Drama Club	R303	Mr.Kiran
Coding Club	Lab1	Mr.Anil

Table 3 Club table after normalization (2NF)

Membership Table:

StudentID (FK)	Club Name (FK)	JoinData
1	Music Club	1/10/2024
1	Sports Club	1/15/2024
2	Sports Club	1/12/2024
2	Drama Club	1/25/2024
3	Music Club	1/20/2024
3	Coding Club	1/28/2024
4	Drama Club	1/18/2024
5	Music Club	1/22/2024
6	Sports Club	1/27/2024
7	Coding Club	1/30/2024

Table 4 Membership table linking students and clubs with join dates

Step 3: Third Normal Form (3NF)

Third Normal Form means the table must already be in 2NF and there should be no transitive dependency. This means that every non-key column should depend only on the primary key and not on other columns. (Silberschatz, A., Korth, H., & Sudarshan, S. (2019))

After converting the database into 2NF, the tables are checked to see if any column depends on another non-key column. In the Student table, the StudentName and Email depend only on StudentID. In the Club table, the ClubRoom and ClubMentor depend only on ClubName. In the Membership table, the JoinDate depends on the combination of StudentID and ClubName.

Since every column depends only on its primary key and not on other columns, there are no transitive dependencies. Therefore, the database now satisfies Third Normal Form (3NF).

Student Table:

StudentID (PK)	Student Name	Email
1	Asha	asha@email.com
2	Bikash	bikash@email.com
3	Nisha	nisha@email.com
4	Rohan	rohan@email.com
5	Suman	suman@email.com
6	Pooja	pooja@email.com
7	Aman	aman@email.com

Table 5 Normalized Student table showing primary key details

Club Table:

ClubName (PK)	Club Room	Club Mentor
Music Club	R101	Mr.Raman
Sports Club	R202	Ms.Sita
Drama Club	R303	Mr.Kiran
Coding Club	Lab1	Mr.Anil

Table 6 Normalized Club table showing primary key details

Membership Table:

StudentID (FK)	Club Name (FK)	JoinData
1	Music Club	1/10/2024
1	Sports Club	1/15/2024
2	Sports Club	1/12/2024
2	Drama Club	1/25/2024
3	Music Club	1/20/2024
3	Coding Club	1/28/2024
4	Drama Club	1/18/2024
5	Music Club	1/22/2024
6	Sports Club	1/27/2024
7	Coding Club	1/30/2024

Table 7 Membership table showing student–club relationships and join dates

Using 1NF, 2NF, and 3NF makes the database better and more organized. It removes repeated data, reduces problems when updating information, and helps keep the data accurate and consistent.

3.3: ER DIAGRAM

The Entity–Relationship (ER) model is used to design the structure of a database. An ER diagram shows the main parts of the system, their details, and how they are connected. In this case, the main entities are Student, Club, and Membership. Student and Club represent real objects, while Membership shows the connection between them. The diagram shows that one student can join many clubs, and one club can have many students. Because of this, the ER model helps in planning and organizing the database before it is actually created. (Chen, P. P.-S. (1976))

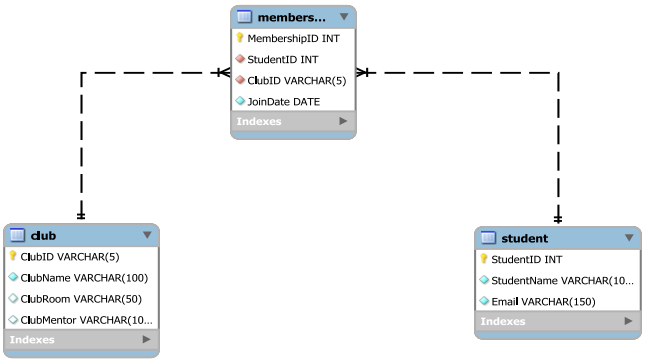


Figure 1 ER diagram of Student, Club, and Membership tables.

a. Entities

An entity is a real-world thing about which we store data in a database. In an ER diagram, an entity is shown using a rectangle. In this diagram, there are three entities: Student, Club, and Membership. The student entity stores details about students, and the Club entity stores details about different clubs. The Membership entity shows the connection between students and clubs. A student can join many clubs, and a club can have many students. Because of this many-to-many relationship, the Membership table is used to link them together. It contains foreign keys that connect the Student and Club tables.

In the given ER diagram, there are three entities: Student, Club, and Membership. The student entity stores information about students, the Club entity stores information about the clubs, and the Membership entity shows the connection between students and clubs.

b. Attributes

In an ER diagram, attributes are the details or information about an entity. They describe the properties of that entity. Some attributes act as primary keys (PK), which are used to uniquely identify each record. Other attributes can be foreign keys (FK), which help connect one entity to another.

In this diagram, there are three entities and each has its own attributes.

The student entity has three attributes: StudentID (PK), StudentName, and Email. The StudentID is the primary key that uniquely identifies each student.

The Club entity has the attributes ClubName (PK), ClubRoom, and ClubMentor. The ClubName acts as the primary key for identifying each club.

The Membership entity includes StudentID (FK), ClubName (FK), and JoinDate. The StudentID and ClubName are foreign keys that connect the Membership table with the Student and Club tables.

Here, PK means Primary Key and FK means Foreign Key.

c. Primary Keys and Foreign Keys

A primary key is used to uniquely identify each record in a table. In this diagram, StudentID is the primary key in the student table, and ClubName is the primary key in the Club table. In the Membership table, the combination of StudentID and ClubName works as a composite primary key because together they uniquely identify each membership record.

A foreign key is used to connect one table to another. It refers to the primary key in another table. In this diagram, StudentID in the Membership table refers to StudentID in the Student table, and ClubName in the Membership table refers to ClubName in the Club table. This helps link students with the clubs they join.

d. Relationship

The ER diagram shows a many-to-many relationship between Student and Club. This means one student can join several clubs, and one club can have many students. To manage this relationship, a Membership entity is used.

A student can join more than one club, and at the same time, a club can include many students. The Membership entity works as a linking table that connects students and clubs. It solves the many-to-many relationship by storing the connection between them.

In the Membership entity, StudentID and ClubName are used as foreign keys to link the Student and Club tables. In the ER diagram, this relationship is shown as one-to-many between Student and Membership, and one-to-many between Club and Membership, which together represent the many-to-many relationship between Student and Club.

In the ER diagram, the crow's foot symbol shows the "many" side of a relationship. The single line shows the "one" side. In this diagram, the crow's foot is on the Membership side, which means one student can have many memberships and one club can also have many memberships. This helps show the many-to-many relationship between Student and Club.

3.4 CONCLUSION

In conclusion, the original table stored all student and club information together, which caused repeated data and problems such as update, insert, and deletion anomalies. This made the database harder to manage and could lead to inconsistent information.

By applying normalization (1NF, 2NF, and 3NF), the data was separated into three tables: Student, Club, and Membership. This removed repeated data and made the structure clearer. The ER diagram also helped show how students and clubs are connected through the Membership table. Overall, normalization makes the database more organized, accurate, and easier to maintain.

4: BASIC SQL OPERATIONS

4.1 SQL QUERIES

The SQL commands below show how to create the normalized tables and do basic database tasks (Oracle, 2024))

```sql


CREATE TABLE Student (
    StudentID   INT          PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL,
    Email       VARCHAR(150) NOT NULL UNIQUE
);

CREATE TABLE Club (
    ClubID     VARCHAR(5)   PRIMARY KEY,
    ClubName   VARCHAR(100) NOT NULL,
    ClubRoom   VARCHAR(50),
    ClubMentor VARCHAR(100)
);

CREATE TABLE Membership (
    MembershipID INT        PRIMARY KEY AUTO_INCREMENT,
    StudentID    INT        NOT NULL,
    ClubID       VARCHAR(5) NOT NULL,
    JoinDate     DATE       NOT NULL,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (ClubID)    REFERENCES Club(ClubID)
);
```

Figure 2 SQL code for creating Student, Club, and Membership tables.

1. Insert a New Student

```
-- Step 6: Add NEW student

INSERT INTO Student (StudentName, Email)
VALUES ('Priya', 'priya@email.com');
```

Figure 3 SQL query to insert a new student into the student table

Adds a new student named Priya with StudentID 8 and her email into the student table.

2. Insert a New Club

```
INSERT INTO Club (ClubID, ClubName, ClubRoom, ClubMentor)
VALUES ('C05', 'Art Club', 'R404', 'Ms. Deepa');
```

Figure 4 SQL query to add a new club to the Club table

Adds Art Club to the database with room R404 and mentor Ms. Deepa

3. Display all the Student

Result Grid | Filter Rows: | Edit: | Export/Import:

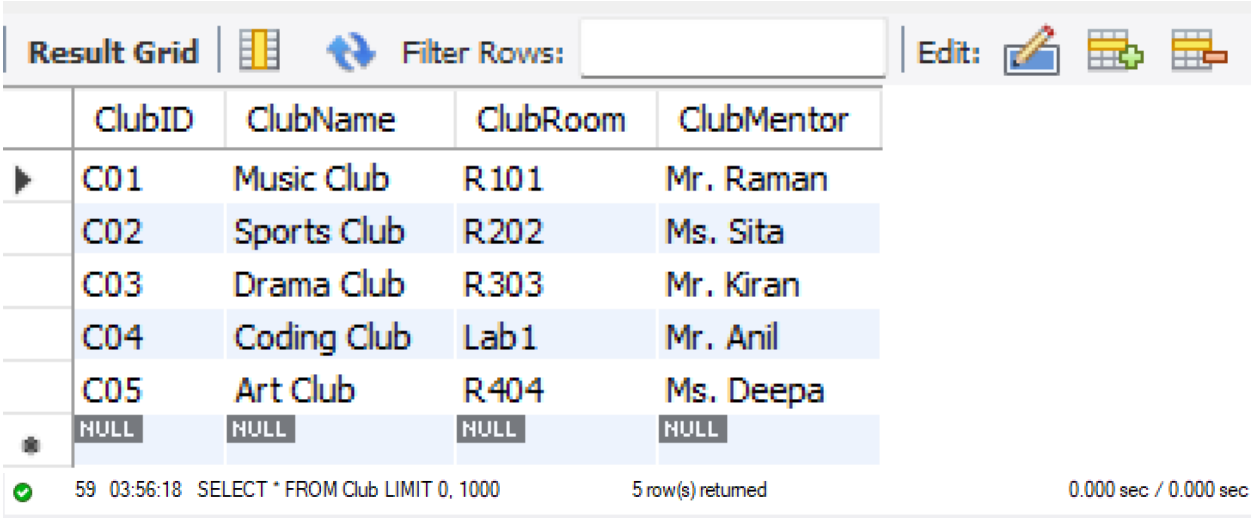
StudentID	StudentName	Email
1	Asha	asha@email.com
2	Bikash	bikash@email.com
3	Nisha	nisha@email.com
4	Rohan	rohan@email.com
5	Suman	suman@email.com
6	Pooja	pooja@email.com
7	Aman	aman@email.com
8	Priya	priya@email.com
NULL	NULL	NULL

58 03:52:42 SELECT * FROM Student LIMIT 0, 1000 8 row(s) returned 0.000 sec / 0.000 sec

Figure 5 Result showing all records in the Student table

Shows all records and columns from the Student table.

4. Display All Code



Result Grid | Filter Rows: | Edit:

ClubID	ClubName	ClubRoom	ClubMentor
C01	Music Club	R101	Mr. Raman
C02	Sports Club	R202	Ms. Sita
C03	Drama Club	R303	Mr. Kiran
C04	Coding Club	Lab1	Mr. Anil
C05	Art Club	R404	Ms. Deepa
NULL	NULL	NULL	NULL

59 03:56:18 SELECT * FROM Club LIMIT 0, 1000 5 row(s) returned 0.000 sec / 0.000 sec

Figure 6 Result showing all records in the Club table

Shows all rows and columns from the Club table.

5: SQL JOIN OPERATION

5.1 DISPLAYING STUDENT NAME, CLUB NAME, AND JOIN DATE

Explanation:

```sql
SELECT
    s.StudentID,
    s.StudentName,
    s.Email,
    c.ClubName,
    c.ClubRoom,
    c.ClubMentor,
    m.JoinDate
FROM
    Membership m
    JOIN Student s ON m.StudentID = s.StudentID
    JOIN Club    c ON m.ClubID    = c.ClubID
ORDER BY
    s.StudentID, m.JoinDate;
```

Figure 7 SQL query using JOIN to combine Student, Club, and Membership data

- FROM Membership m – starts with the Membership table.
- JOIN Student s ON m.StudentID = s.StudentID – connects the membership with the correct student.
- JOIN Club c ON m.ClubID = c.ClubID – connects the membership with the correct club.
- ORDER BY – sorts the results by student name and then by date.

Result:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

StudentID	StudentName	Email	ClubName	ClubRoom	ClubMentor	JoinDate
1	Asha	asha@email.com	Music Club	R101	Mr. Raman	2024-01-1
1	Asha	asha@email.com	Music Club	R101	Mr. Raman	2024-01-1
1	Asha	asha@email.com	Music Club	R101	Mr. Raman	2024-01-1
1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	2024-01-1
1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	2024-01-1
1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	2024-01-1
2	Bikash	bikash@email.com	Music Club	R101	Mr. Raman	2024-01-2
2	Bikash	bikash@email.com	Music Club	R101	Mr. Raman	2024-01-2
2	Bikash	bikash@email.com	Music Club	R101	Mr. Raman	2024-01-2
2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	2024-01-2
2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	2024-01-2
2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	2024-01-2
3	Nisha	nisha@email.com	Sports Club	R202	Ms. Sita	2024-01-1
3	Nisha	nisha@email.com	Sports Club	R202	Ms. Sita	2024-01-1
3	Nisha	nisha@email.com	Sports Club	R202	Ms. Sita	2024-01-1

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

StudentID	StudentName	Email	ClubName	ClubRoom	ClubMentor	JoinDate
3	Nisha	nisha@email.com	Coding Club	Lab1	Mr. Anil	2024-01-2
4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	2024-01-2
4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	2024-01-2
4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	2024-01-2
5	Suman	suman@email.com	Music Club	R101	Mr. Raman	2024-01-1
5	Suman	suman@email.com	Music Club	R101	Mr. Raman	2024-01-1
5	Suman	suman@email.com	Music Club	R101	Mr. Raman	2024-01-1
6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	2024-01-3
6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	2024-01-3
6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	2024-01-3
7	Aman	aman@email.com	Coding Club	Lab1	Mr. Anil	2024-01-2
7	Aman	aman@email.com	Coding Club	Lab1	Mr. Anil	2024-01-2
7	Aman	aman@email.com	Coding Club	Lab1	Mr. Anil	2024-01-2
8	Priya	priya@email.com	Music Club	R101	Mr. Raman	2024-02-0

Figure 8Result showing combined student, club, and membership data using a JOIN query

6: REFLECTION QUESTION

6.1 EXPLAIN HOW NORMALIZATION

a. Reduces repeated data

Normalization helps reduce repeated data by storing information in separate tables. For example, student details are stored only in the student table, and club details are stored only in the Club table. This avoids repeating the same information many times.

b. Makes the database easier to manage

Normalization keeps data more accurate because each piece of information is stored in only one place. If a student's email or a club's room changes, it only needs to be updated once.

6.2 NEED FOR JOIN AFTER NORMALIZATION

After normalization, data is stored in different tables. To see complete information, the tables must be connected using JOIN. This helps combine data from Student, Club, and Membership tables to show full results.

7. CONCLUSION

This report explained important computer science concepts related to data and systems. It first discussed encoding formats such as Base64, Hex, ASCII, and URL encoding and how they help transfer data across networks. The report also explained that encoding alone does not provide security, so it must work together with encryption methods like TLS.

The report also examined the classroom seating problem using the ideas of P and NP problems. It showed that checking a seating arrangement is easy, but finding the correct arrangement can be difficult because the number of possibilities grows quickly. Finally, the report explained database normalization and showed how dividing data into proper tables improves accuracy, reduces repetition, and keeps the database organized. Overall, these concepts help build secure, efficient, and reliable computer systems.